

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Инженерия Искусственного Интеллекта

Домашняя работа № 3

Дисциплина: Фронтенд разработка

“CSS-переменные и темизация сайта”

Выполнил:

Фиолетов Эдуард Анджеевич

Группа J3213

Проверил:

Добряков Д. Ю.

Санкт-Петербург

2026 г.

Содержание

1. Цель работы	3
2. Задание	3
3. Обоснование стека проекта	3
4. Используемые файлы проекта	3
5. CSS-переменные	3
6. Темная тема	4
7. Интеграция с Tailwind	4
8. JavaScript-переключение темы	4
9. Сохранение выбранной темы	5
10. Использование темы в интерфейсе	5
11. Проверка результата	6
12. Вывод	6

1. Цель работы

Цель домашней работы - реализовать темизацию сайта с использованием CSS-переменных, добавить несколько тем оформления и обеспечить сохранение выбранной темы для пользователя.

Работа выполнена на Marketplace Alarm Bot 2. Для операционного интерфейса важны две задачи: светлая тема для обычной офисной работы и темная тема для длительной работы с заказами. Темизация вынесена на уровень CSS-переменных, поэтому компоненты не зависят от конкретных цветов.

2. Задание

По заданию необходимо:

- добавить к сайту дополнительную тему;
- реализовать переключение темы через JavaScript;
- использовать CSS-переменные для цветов и визуальных токенов;
- сохранить единый набор компонентов при смене темы;
- проверить, что интерфейс корректно отображается в обеих темах.

3. Обоснование стека проекта

В проекте используется Tailwind CSS 4 и CSS custom properties. Tailwind отвечает за компоновку и применение utility-классов, а CSS-переменные задают дизайн-токены: фон, текст, карточки, границы, состояния, цвета маркетплейсов и цвета графиков. Поэтому для новой темы не приходится дублировать классы компонентов.

JavaScript-часть темизации реализована в React-контексте ThemeProvider. Он читает сохраненную тему из localStorage, добавляет классы темы на элемент html и сохраняет выбор пользователя.

4. Используемые файлы проекта

- ./web/frontend/src/index.css - CSS-переменные, светлая и темная темы
- ./web/frontend/src/context/ThemeContext.tsx - логика выбора и сохранения темы
- ./web/frontend/src/main.tsx - подключение ThemeProvider
- ./web/frontend/src/pages/HealthPage.tsx - переключение темы в интерфейсе

5. CSS-переменные

Основные цвета объявлены в :root. Они описывают не конкретные элементы, а смысловые роли: фон, текст, карточка, popover, primary, secondary, muted, border, input и ring.

```
:root {
  --background: rgb(255, 255, 255);
  --foreground: rgb(10, 10, 10);
  --card: rgb(255, 255, 255);
  --card-foreground: rgb(10, 10, 10);
  --primary: rgb(23, 23, 23);
  --primary-foreground: rgb(250, 250, 250);
```

```

--border: rgb(229, 229, 229);
--ring: rgb(161, 161, 161);
}

```

Также объявлены доменные цвета маркетплейсов:

```

--mp-wb: #9b59b6;
--mp-ozon: #005bff;
--mp-yandex: #ff7300;

```

Поэтому бейджи, таблицы и навигация используют одни и те же цвета площадки во всем приложении.

6. Темная тема

Темная тема задается классом `.dark`. В нем переопределяются те же переменные, что и в `:root`: фон становится темным, текст - светлым, границы и карточки получают более контрастные значения.

```

.dark {
  --background: rgb(10, 10, 10);
  --foreground: rgb(250, 250, 250);
  --card: rgb(23, 23, 23);
  --card-foreground: rgb(250, 250, 250);
  --border: rgb(40, 40, 40);
}

```

Компоненты при этом не меняются. Например, класс `bg-card text-card-foreground` автоматически получает правильные значения в зависимости от активной темы.

7. Интеграция с Tailwind

В `index.css` используется блок `@theme inline`, который связывает CSS-переменные с Tailwind-токенами. После этого в компонентах можно использовать классы `bg-background`, `text-foreground`, `border-border`, `bg-card`, `text-muted-foreground`.

```

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --color-card: var(--card);
  --color-border: var(--border);
  --color-ring: var(--ring);
}

```

Компоненты остаются декларативными, а тема меняется централизованно.

8. JavaScript-переключение темы

Логика переключения находится в `ThemeContext.tsx`. Доступны две темы:

```

export const THEMES = [
  { id: 'linear', label: 'Linear', dark: true },
  { id: 'light', label: 'Светлая', dark: false },
];

```

При изменении темы провайдер обновляет классы на `document.documentElement`. Если тема темная, добавляется класс `dark`, который активирует темные CSS-переменные.

```
useEffect(() => {
  const html = document.documentElement;
  html.classList.remove('light', 'linear', 'dark');
  html.classList.add(theme);
  if (THEMES.find((t) => t.id === theme)?.dark) html.classList.add('dark');
  localStorage.setItem(storageKey(uid), theme);
}, [theme, uid]);
```

9. Сохранение выбранной темы

Выбор сохраняется в `localStorage`. Для авторизованного пользователя используется ключ с `uid`, поэтому разные пользователи на одном браузере могут иметь разные предпочтения.

```
function storageKey(uid?: string) {
  return uid ? `mab_theme_${uid}` : 'mab_theme';
}

function readTheme(uid?: string) {
  const saved = localStorage.getItem(storageKey(uid)) ??
  localStorage.getItem('mab_theme');
  return THEMES.find((t) => t.id === saved)?.id ?? 'linear';
}
```

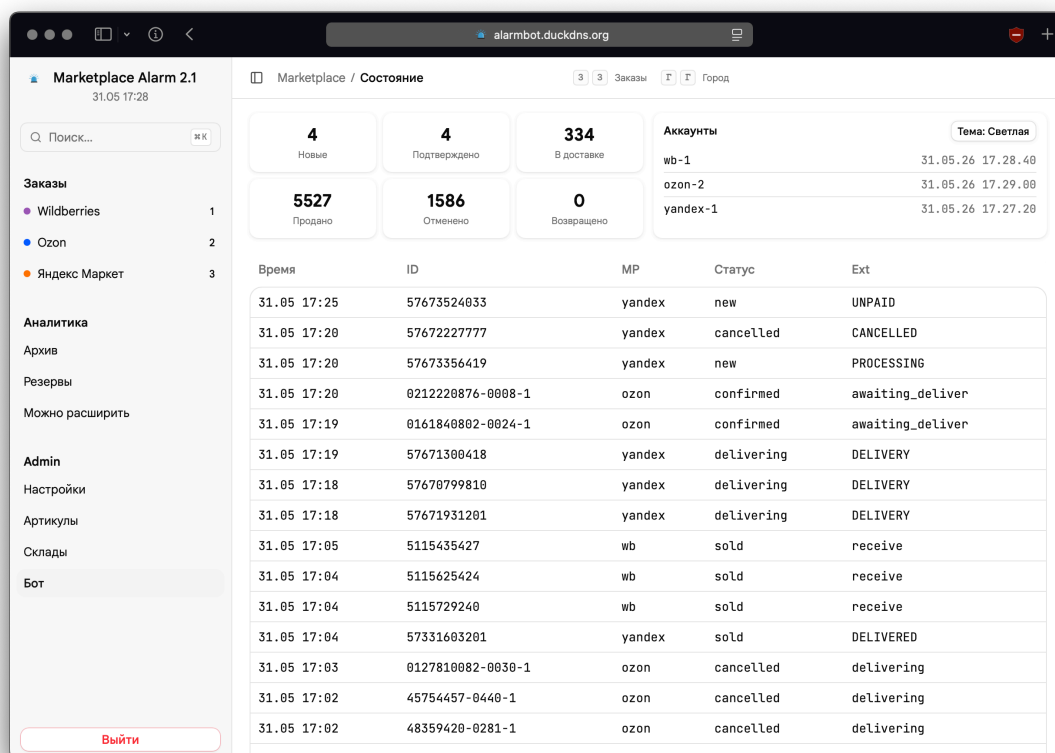
10. Использование темы в интерфейсе

На странице состояния сервиса пользователь может переключить тему. После выбора новая тема применяется без перезагрузки страницы, а значение сохраняется в браузере.

Компоненты используют смысловые классы:

- `bg-background` - фон приложения;
- `text-foreground` - основной текст;
- `bg-card` - карточки и панели;
- `text-muted-foreground` - второстепенный текст;
- `border-border` - границы;
- `ring-ring` - фокус.

Один и тот же компонент за счет этого корректно выглядит в светлой и темной теме.



Страница состояния сервиса в светлой теме: переключатель темы, карточки счетчиков, аккаунты и журнал событий.

11. Проверка результата

Проверка выполнялась по шагам:

1. Открыть приложение в светлой теме.
2. Перейти на главную страницу, список заказов и доставку.
3. Переключить тему на темную.
4. Проверить фон, карточки, таблицы, меню, формы и модальные окна.
5. Обновить страницу и убедиться, что выбранная тема сохранена.
6. Проверить цвета маркетплейсов в обеих темах.
7. Проверить видимость фокуса и контраст текста.

12. Вывод

В работе описана система темизации Marketplace Alarm Bot 2. Цвета и визуальные токены вынесены в CSS-переменные, темы переключаются через JavaScript и сохраняются в localStorage. Компоненты используют смысловые классы и не зависят от конкретных цветовых значений, поэтому светлое и темное оформление работают без дублирования разметки.